

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESCUELA DE CIENCIAS Y SISTEMAS
LENGUAJES FORMALES Y DE PROGRAMACIÓN

HORARIOSCRIPT

Proyecto 1 — Analizador Léxico para Horarios Académicos
Manual Técnico

Josué Eliú Castro Sosa
Carné: 202405110
Segundo Semestre 2026
Guatemala, Septiembre 2026

1. Introducción

HorarioScript es un analizador léxico hecho en Python 3.10+ con interfaz gráfica en Tkinter. Lee archivos .hor y reconoce sus tokens con un Autómata Finito Determinista (AFD) escrito a mano, sin usar el módulo re ni generadores automáticos de analizadores. Este documento explica cómo está organizado el código, cómo funciona el AFD, cómo se detectan los choques de horario, y por qué se tomaron ciertas decisiones durante el desarrollo.

2. Arquitectura del Sistema

El proyecto se separó en varios módulos, cada uno encargado de una sola cosa: AnalizadorLexico hace la tokenización, Token representa cada token individual, GestorErrores y ErrorLexico acumulan los errores léxicos, estructura.py y detector_choques.py arman los datos de cursos, catedráticos, aulas y clases a partir de los tokens ya generados, GeneradorReportes construye los 3 reportes HTML, exportador_dot.py genera el diagrama del AFD en formato DOT, y VentanaHorarioScript es la GUI que conecta todo lo anterior con botones y tablas.

Separarlo así facilitó el desarrollo: cada archivo se pudo probar por separado desde consola antes de conectarlo a la interfaz gráfica, y cuando algo fallaba era más rápido encontrar en qué módulo estaba el problema. La GUI no hace cálculos ni valida nada por su cuenta: solo llama a las funciones de los demás archivos y muestra el resultado.

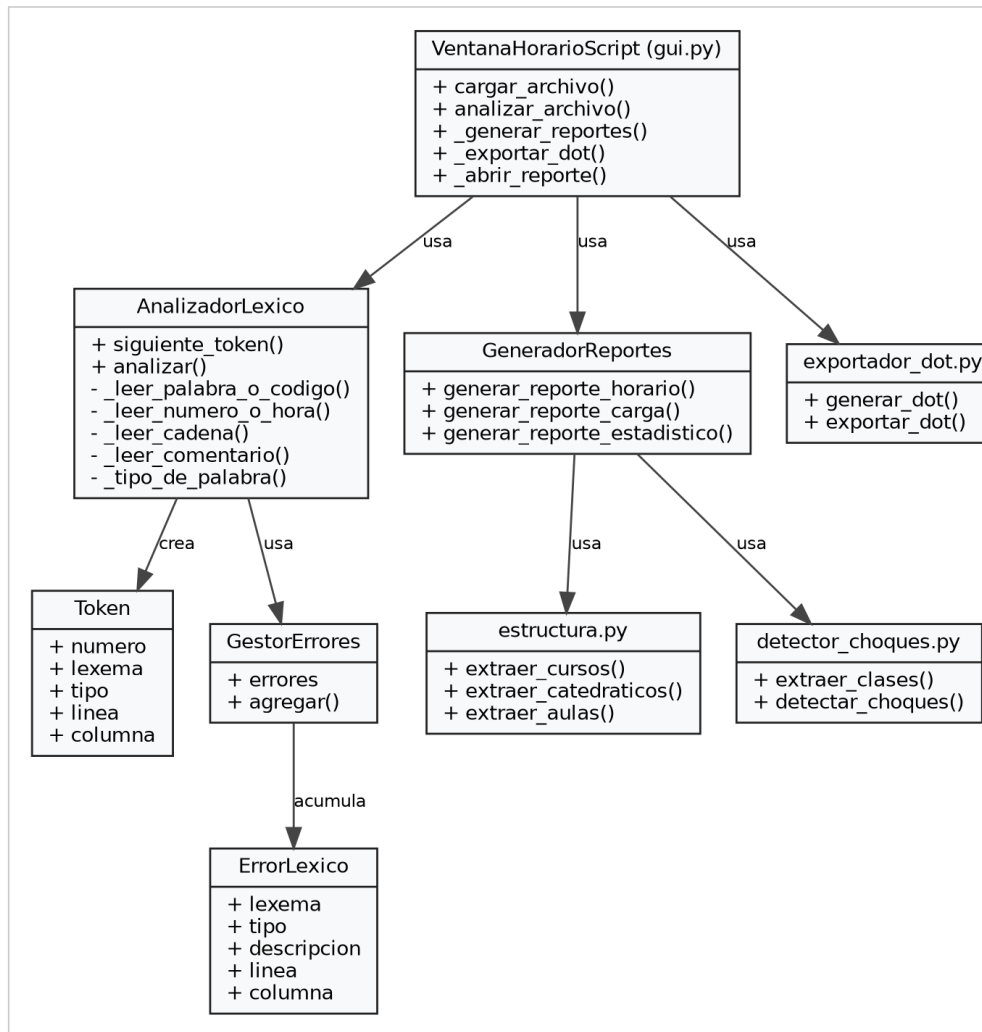


Figura 1. Diagrama de clases y módulos del sistema HorarioScript.

Flujo real de ejecución: la GUI recibe la ruta del archivo .hor seleccionado, crea un AnalizadorLexico con el texto leído, y obtiene la lista de tokens junto con los errores acumulados. Con esos mismos tokens, GeneradorReportes reconstruye cursos, catedráticos, aulas y clases, detecta los choques, y arma los 3 HTML.

3. Diagrama del Autómata Finito Determinista

El AFD está escrito a mano con `if/elif` dentro de los métodos de `AnalizadorLexico`. El analizador recorre el texto carácter por carácter usando indexación directa (`texto[i]`), sin `re` y sin funciones de alto nivel de cadenas para tokenizar. El diagrama de la Figura 2 no se dibujó a mano: se generó con una función Python (`exportador_dot.py`) que recorre una tabla de transiciones y escribe el código DOT para Graphviz.

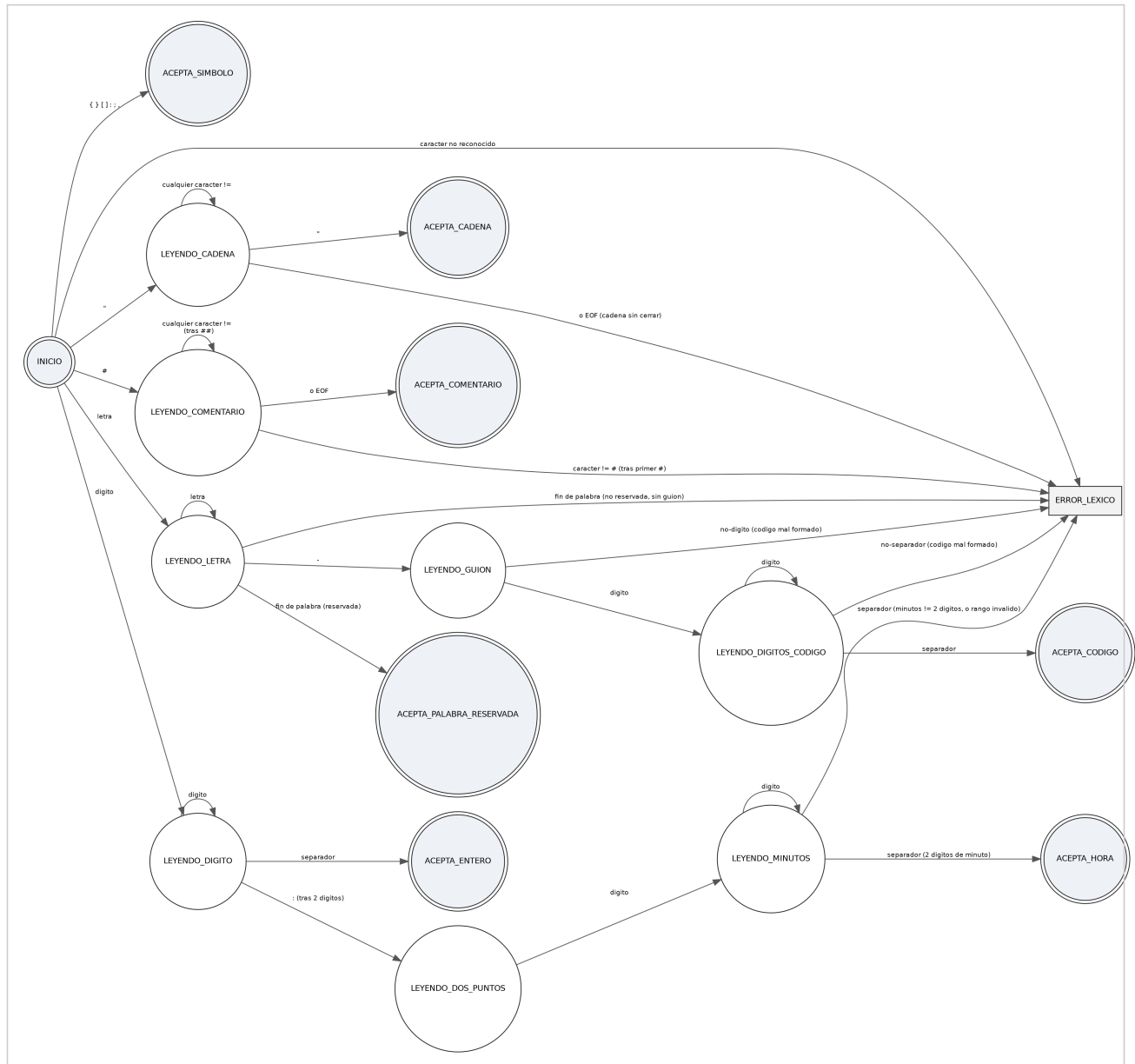


Figura 2. Diagrama de estados del AFD de HorarioScript, generado automáticamente a partir de la tabla de transiciones del sistema.

Los 5 tipos de error del enunciado (`CARACTER_NO_RECONOCIDO`, `CADENA_SIN_CERRAR`, `HORA_FUERA_DE_RANGO`, `DIA_NO_RECONOCIDO`, `CODIGO_MAL_FORMADO`) se agrupan en

un solo estado `ERROR_LEXICO` dentro del diagrama. Se hizo así para no saturar el grafo con flechas repetidas: dibujar los 5 tipos por separado hubiera hecho el diagrama más difícil de leer sin aportar nada nuevo, porque el camino que recorre el AFD es el mismo en los 5 casos y solo cambia el mensaje que se guarda. En el código real cada error sí se distingue: `GestorErrores.agregar ( )` guarda el tipo específico, como se explica en la Sección 5.

4. Tabla de Transiciones

Transiciones tal como las genera exportador_dot.py:

Estado origen	Estado destino	Símbolo / condición
INICIO	LEYENDO_LETRA	letra
LEYENDO_LETRA	LEYENDO_LETRA	letra
LEYENDO_LETRA	LEYENDO_GUION	-
LEYENDO_LETRA	ACEPTA_PALABRA_RESERVADA	fin de palabra (reservada)
LEYENDO_LETRA	ERROR_LEXICO	fin de palabra (no reservada, sin guion)
LEYENDO_GUION	LEYENDO_DIGITOS_CODIGO	digito
LEYENDO_GUION	ERROR_LEXICO	no-digito (codigo mal formado)
LEYENDO_DIGITOS_CODIGO	LEYENDO_DIGITOS_CODIGO	digito
LEYENDO_DIGITOS_CODIGO	ACEPTA_CODIGO	separador
LEYENDO_DIGITOS_CODIGO	ERROR_LEXICO	no-separador (codigo mal formado)
INICIO	LEYENDO_DIGITO	digito
LEYENDO_DIGITO	LEYENDO_DIGITO	digito
LEYENDO_DIGITO	LEYENDO_DOS_PUNTOS	: (tras 2 digitos)
LEYENDO_DIGITO	ACEPTA_ENTERO	separador
LEYENDO_DOS_PUNTOS	LEYENDO_MINUTOS	digito
LEYENDO_MINUTOS	LEYENDO_MINUTOS	digito
LEYENDO_MINUTOS	ACEPTA_HORA	separador (2 digitos minuto)
LEYENDO_MINUTOS	ERROR_LEXICO	separador (minutos invalidos / rango)
INICIO	LEYENDO_CADENA	"
LEYENDO_CADENA	LEYENDO_CADENA	caracter != \n
LEYENDO_CADENA	ACEPTA_CADENA	"
LEYENDO_CADENA	ERROR_LEXICO	\n o EOF (sin cerrar)
INICIO	LEYENDO_COMENTARIO	#
LEYENDO_COMENTARIO	ERROR_LEXICO	caracter != # (tras primer #)
LEYENDO_COMENTARIO	LEYENDO_COMENTARIO	caracter != \n (tras ##)
LEYENDO_COMENTARIO	ACEPTA_COMENTARIO	\n o EOF
INICIO	ACEPTA_SIMBOLO	{ } [] : ; ,
INICIO	ERROR_LEXICO	caracter no reconocido

5. Cómo Funciona la Tokenización

`siguiente_token()` es el método principal. Cada vez que se llama: (1) salta espacios y saltos de línea con `_ignorar_espacios()`, actualizando línea y columna; (2) revisa si ya se llegó al final del archivo; (3) mira el primer carácter que falta procesar y, según qué tipo de carácter es, manda el control a uno de cuatro métodos.

5.1 Palabras reservadas y códigos

`_leer_palabra_o_codigo()` resuelve la ambigüedad entre palabra reservada y código consumiendo primero todas las letras, y decidiendo el tipo hasta el final. Si después de las letras viene un guion, se asume que es un CÓDIGO y se exige al menos un dígito después; si no hay dígito, se marca `CODIGO_MAL_FORMADO`. Si no hay guion, la palabra se busca en los diccionarios `BLOQUES`, `ELEMENTOS`, `ATRIBUTOS`, `RELACIONES`, `DIAS` y `CATEGORIAS`. Usar diccionarios aquí es más rápido que comparar contra una lista larga de `if`, porque buscar una clave en un diccionario de Python es prácticamente instantáneo sin importar cuántas palabras reservadas haya.

5.2 HORA y ENTERO

`_leer_numero_o_hora()` resuelve la otra ambigüedad del lenguaje: si tras leer dígitos se encontraron exactamente dos y sigue un `:`, se asume HORA y se exigen exactamente dos dígitos más para los minutos. El rango 06:00–21:00 se valida aparte, en `_hora_valida()`, una vez que ya se tiene el lexema completo. Si no se cumple el patrón de hora, se toma como ENTERO.

5.3 CADENA y COMENTARIO_LINEA

`_leer_cadena()` avanza carácter por carácter después de la comilla de apertura hasta encontrar otra comilla o un salto de línea; si llega al salto de línea sin cerrar, se marca `CADENA_SIN_CERRAR` guardando la posición donde abrió. `_leer_comentario()` reconoce `##` y se come el resto de la línea completa sin fijarse en qué caracteres trae, porque el enunciado dice que un comentario nunca genera error aunque tenga símbolos raros adentro.

5.4 Modo pánico (recuperación de errores)

Cuando se detecta algo inválido, se llama a `GestorErrores.agregar()` con el lexema, el tipo de error, la descripción, línea y columna, y luego se sigue con `siguiente_token()` (o `_recuperar()`, que avanza hasta el próximo separador) para no cortar el análisis ahí. Con esto el programa reporta todos los errores del archivo de una sola pasada en vez de detenerse en el primero que encuentra.

6. Cómo se Detectan los Choques de Horario

`detector_choques.py` trabaja en dos pasos. Primero, `extraer_clases()` recorre la lista de tokens que ya generó el AFD y arma cada bloque `clase` como un diccionario con curso, catedrático, aula, día, hora de inicio, hora de fin y sección. Segundo, `detectar_choques()` compara cada par de clases entre sí (complejidad $O(n^2)$, que no es problema para el tamaño normal de un archivo de horario) y marca un choque cuando se cumplen a la vez: mismo día, bloques de horario que se traslapan, y mismo catedrático o misma aula.

```
# Verificacion de traslape y coincidencia
se_traslapan = (inicio1 < fin2) and (inicio2 < fin1)
mismo_catedratico = (clase1["catedratico"] == clase2["catedratico"])
mismo_aula = (clase1["aula"] == clase2["aula"])

if se_traslapan and (mismo_catedratico or mismo_aula):
    # Se registra el choque de horario
```

7. Decisiones de Diseño

7.1 Consumir primero, clasificar después

En vez de intentar adivinar el tipo de token desde el primer carácter, el analizador consume toda la palabra y decide al final. Esto evita tener que crear un estado distinto para cada palabra reservada del lenguaje: todas pasan por el mismo camino (`LEYENDO_LETRA`), y clasificarlas al final con un diccionario es mucho más simple de mantener que ir armando el AFD palabra por palabra.

7.2 Agrupar los errores en un solo estado del diagrama

Internamente el sistema sí distingue los 5 tipos de error, cada uno con su propio mensaje. Pero en el diagrama se dibujan todos yendo al mismo estado `ERROR_LEXICO` para que el grafo no quede lleno de flechas repetidas que no aportan nada nuevo visualmente — el camino que recorre el AFD es el mismo en los 5 casos, lo único que cambia es el mensaje guardado en `GestorErrores`.

7.3 Capacidad de 90 horas semanales por aula

El Reporte 3 calcula el % de ocupación de cada aula sobre 90 horas: 15 horas diarias (06:00 a 21:00) por 6 días hábiles (LUNES a SÁBADO). El enunciado no dice qué contar como “bloque disponible”, así que se tomó este número como aproximación razonable. Se puede ajustar cambiando la constante `CAPACIDAD_SEMANAL_AULA_HORAS` en `generador_reportes.py`.

7.4 Umbrales de carga de catedráticos

Los niveles BAJA (1–4 horas), NORMAL (5–10), ALTA (11–15) y SATURADA (16+) se tomaron directo del enunciado sin cambiarlos, porque ya son valores razonables para una carga académica normal en un semestre.

7.5 Separar el proyecto en varios archivos

`estructura.py`, `detector_choques.py` y `generador_reportes.py` se mantuvieron como archivos separados en vez de meter todo en uno solo. Así `GeneradorReportes` pudo reutilizar directamente las funciones de choques y de extracción de datos sin repetir código, y cada parte se pudo probar y depurar por separado durante el desarrollo.

7.6 Por qué usar `.strip()` no viola la regla de tokenización

Los reportes usan `.strip()` para quitarle las comillas a una CADENA ya reconocida (por ejemplo, pasar de `"LFP-0796"` a `LFP-0796` antes de mostrarlo en un reporte). Esto pasa después de que el AFD ya tokenizó todo el archivo carácter por carácter; no decide dónde empieza o termina un token, así que no cuenta como usar funciones de alto nivel para tokenizar — es solo limpieza de datos ya procesados.